

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №5
по дисциплине
Интеграция и тестирование программных систем
по теме
«Автоматизация UI-тестирования»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял:

Басалов Ю.А.

Тула 2026

Лабораторная работа №5

по теме «Автоматизация UI-тестирования»

Цель работы

Получить практические навыки автоматизации тестирования пользовательского интерфейса с использованием Selenium WebDriver: освоить паттерн Page Object Model (POM) для создания поддерживаемых тестов, явные и неявные ожидания для устойчивости к разной скорости загрузки, обработку исключений, кросс-браузерный запуск (Chrome, Firefox, режим headless), фикстуры PyTest с автоматическими скриншотами упавших тестов и отчётность (pytest-html, JUnit XML, Allure).

Описание тестируемого веб-приложения (вариант 9)

Объект тестирования — публичный демонстрационный сайт The Internet (<https://the-internet.herokuapp.com/>), вариант 9: разделы Geolocation, Inputs и Notification Messages. Требование задания — не менее 3 автоматизированных тестовых сценариев; фактически автоматизировано 16 (7 — Inputs, 5 — Notification Messages, 4 — Geolocation).

Раздел	URL	Функциональность
Главная	/	список ссылок на все разделы (входная точка навигации)
Inputs	/inputs	единственное поле ввода типа number: ввод числа, очистка, стрелки ↑/↓ меняют значение на единицу
Notification Messages	/notification_message	ссылка «Click here» перезагружает страницу со случайным flash-уведомлением («Action successful» либо «Action unsuccessful, please try again» — опечатка на самом сайте); уведомление закрывается крестиком
Geolocation	/geolocation	кнопка «Where am I?» запрашивает navigator.geolocation и выводит широту/долготу со ссылкой «See it on Google»

Welcome to the-internet

Available Examples

[A/B Testing](#)
[Add/Remove Elements](#)
[Basic Auth](#) (user and pass: admin)
[Broken Images](#)
[Challenging DOM](#)
[Checkboxes](#)
[Context Menu](#)
[Digest Authentication](#) (user and pass: admin)
[Disappearing Elements](#)
[Drag and Drop](#)

Рисунок 1 — Главная страница The Internet со списком разделов

Inputs

Number

Powered by [Elemental Selenium](#)

Рисунок 2 — Раздел Inputs: поле ввода числа со введённым значением



❗ Action unsuccessful, please try again

Notification Message

The message displayed above the heading is a notification message. It is often used to convey information about an action previously taken by the user.

Some rudimentary examples include 'Action successful', 'Action unsuccessful, please try again', etc.

[Click here](#) to load a new message.

Powered by [Elemental Selenium](#)

Рисунок 3 — Раздел Notification Messages: flash-уведомление после клика «Click here»

Geolocation



Latitude:
55.75

Longitude:
37.6167

[See it on Google](#)

Where am I?

Powered by [Elemental Selenium](#)

Рисунок 4 — Раздел Geolocation: координаты и ссылка на карту после клика «Where am I?»

Тестовые сценарии и критерии успешности

Сценарии спроектированы по разделам сайта и включают позитивные, негативные и навигационные проверки. Критерий успешности каждого сценария — совпадение фактического состояния элемента (значение поля, видимость и текст уведомления, выведенные координаты, URL после перехода) с ожидаемым.

ID	Сценарий (критерий успешности)
TC-INP-01	Переход в раздел Inputs с главной страницы (URL /inputs, заголовок «Inputs») [smoke]
TC-INP-02	Ввод целого числа 42 (значение поля — «42») [smoke]
TC-INP-03	Ввод десятичного числа 3.14 (значение — «3.14»)
TC-INP-04	Ввод отрицательного числа -5 (значение — «-5»)
TC-INP-05	Очистка заполненного поля (значение — пустая строка)
TC-INP-06	Негативный: нечисловой ввод «abc» в поле type=number игнорируется (значение — пустая строка)
TC-INP-07	Стрелки ↑/↓ изменяют значение на единицу (0 → 1 → 2 → 1)
TC-NOT-01	Переход в раздел Notification Messages с главной страницы [smoke]
TC-NOT-02	После клика «Click here» отображается flash-уведомление [smoke]
TC-NOT-03	Текст уведомления — один из двух ожидаемых вариантов сайта
TC-NOT-04	Негативный: уведомление закрывается крестиком и исчезает со страницы
TC-NOT-05	Каждый из трёх последовательных кликов «Click here» загружает новое уведомление с ожидаемым текстом
TC-GEO-01	Переход в раздел Geolocation с главной страницы [smoke]
TC-GEO-02	До клика выводится приглашение нажать кнопку
TC-GEO-03	Кнопка «Where am I?» выводит заданные координаты (стаб geolocation) [smoke]
TC-GEO-04	Ссылка «See it on Google» содержит выведенные координаты (q=широта,долгота)

Структура тестового проекта

Проект размещён в ЛР5\ui_tests (базой послужил стартовый каркас из материалов Lab 5, переработанный под the-internet.herokuapp.com: заменены Page Objects, дописаны отсутствовавшие utils/logger.py, utils/screenshot.py, data/test_data.py, параметры --browser/--headless). Назначение файлов:

Файл / каталог	Назначение
pages/base_page.py	BasePage: поиск элементов, клики (с JS-fallback), ввод текста, явные ожидания, логирование, скриншоты
pages/home_page.py	Page Object главной страницы: заголовок, переходы по ссылкам в разделы
pages/inputs_page.py	Page Object раздела Inputs: ввод, очистка, клавиши (в т. ч. стрелки)
pages/notification_messages_page.py	Page Object раздела Notification Messages: клик «Click here», чтение и закрытие уведомления
pages/geolocation_page.py	Page Object раздела Geolocation: стаб navigator.geolocation, кнопка «Where am I?», чтение координат и ссылки на карту
tests/conftest.py	фикстуры: параметры --browser/--headless, driver (создание и закрытие драйвера), страницы, autouse-скриншот при падении, хук результатов теста
tests/test_inputs.py	7 тестов раздела Inputs (TC-INP-01...07)
tests/test_notification_messages.py	5 тестов раздела Notification Messages (TC-NOT-01...05)
tests/test_geolocation.py	4 теста раздела Geolocation (TC-GEO-01...04)
utils/driver_manager.py	DriverManager: Chrome/Firefox/Edge, режим headless, автоматическая загрузка драйверов webdriver-manager
utils/logger.py	настройка логирования: консоль + файл logs/ui_tests.log
utils/screenshot.py	ScreenshotManager: скриншоты в screenshots/ с меткой времени
data/test_data.py	TestData: базовый URL, тестовые значения, тексты уведомлений, координаты стаба
pytest.ini	конфигурация: testpaths, pythonpath, маркеры, addopts (pytest-html, JUnit XML, автоповтор падений)
requirements.txt	зависимости проекта (selenium, pytest, webdriver-manager, allure-pytest, pytest-html, pytest-xdist, pytest-rerunfailures)
run_tests.py	лаунчер прогонов: полный, smoke, по разделам, Firefox, Allure
make_site_screenshots.py	вспомогательный скрипт съёмки скриншотов сайта для отчёта

Управление веб-драйвером (utils/driver_manager.py)

Класс DriverManager создаёт драйвер выбранного браузера (chrome, firefox, edge — параметр --browser) с автоматической загрузкой подходящего драйвера через webdriver-manager. Для Chrome и Edge в headless-режиме используется новый headless-режим (--headless=new), для Firefox — аргумент --headless; окно фиксированного размера 1920×1080, неявное ожидание 10 секунд. Класс также инкапсулирует корректное закрытие драйвера (quit_driver) и скриншоты.

```
"""
```

```
Управление веб-драйвером (Chrome / Firefox / Edge, режим headless).
```

```
Автоматическая загрузка драйверов через webdriver-manager.
```

```
"""
```

```
import logging
from typing import Optional
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.firefox.service import Service as FirefoxService
from selenium.webdriver.edge.service import Service as EdgeService
from webdriver_manager.chrome import ChromeDriverManager
from webdriver_manager.firefox import GeckoDriverManager
from webdriver_manager.microsoft import EdgeChromiumDriverManager
```

```
logger = logging.getLogger(__name__)
```

```
class DriverManager:
```

```
    """Класс для управления веб-драйвером"""
```

```
    def __init__(self, browser: str = "chrome", headless: bool = False):
```

```
        self.browser = browser.lower()
```

```
        self.headless = headless
```

```

self.driver: Optional[webdriver.Remote] = None

def create_driver(self) -> webdriver.Remote:
    """Создание экземпляра веб-драйвера"""
    logger.info(f"Создание драйвера для браузера:
{self.browser}")

    if self.browser == "chrome":
        options = webdriver.ChromeOptions()
        if self.headless:
            options.add_argument("--headless=new")
            options.add_argument("--no-sandbox")
            options.add_argument("--disable-dev-shm-usage")
            options.add_argument("--window-size=1920,1080")
            options.add_argument("--disable-notifications")
            options.add_experimental_option("excludeSwitches",
["enable-logging"])

        service =
ChromeService(ChromeDriverManager().install())
        self.driver = webdriver.Chrome(service=service,
options=options)

    elif self.browser == "firefox":
        options = webdriver.FirefoxOptions()
        if self.headless:
            options.add_argument("--headless")
            options.add_argument("--width=1920")
            options.add_argument("--height=1080")

        service =
FirefoxService(GeckoDriverManager().install())
        self.driver = webdriver.Firefox(service=service,
options=options)

    elif self.browser == "edge":
        options = webdriver.EdgeOptions()
        if self.headless:

```

```

        options.add_argument("--headless=new")
        options.add_argument("--no-sandbox")
        options.add_argument("--disable-dev-shm-usage")

        service =
EdgeService(EdgeChromiumDriverManager().install())

        self.driver = webdriver.Edge(service=service,
options=options)

    else:

        raise ValueError(f"Неподдерживаемый браузер:
{self.browser}")

    # Настройки драйвера
    self.driver.implicitly_wait(10) # Неявное ожидание
    self.driver.maximize_window()

    logger.info(f"Драйвер создан: {self.browser}
(headless={self.headless})")
    return self.driver

def quit_driver(self):
    """Закрытие драйвера"""
    if self.driver:
        logger.info("Закрытие драйвера")
        self.driver.quit()
        self.driver = None

def take_screenshot(self, filename: str = "screenshot.png"):
    """Создание скриншота текущей страницы"""
    if self.driver:
        self.driver.save_screenshot(filename)
        logger.info(f"Скриншот сохранен: {filename}")
        return filename
    return None

```

Базовый класс страницы (pages/base_page.py)

BasePage инкапсулирует работу с элементами: поиск с явными ожиданиями (presence/visibility/clickable), клики, ввод текста, чтение текста и атрибутов, скриншоты — каждое действие логируется. Ключевая деталь устойчивости: если обычный клик перехватывается другим элементом (ElementClickInterceptedException) или элемент временно недоступен, выполняется повторный клик через JavaScript (execute_script) — это спасает тесты от враждебной вёрстки (см. анализ проблем).

```
# -*- coding: utf-8 -*-
"""
Базовый класс для всех страниц (the-internet.herokuapp.com).
Поиск элементов, клики, ввод текста, явные ожидания,
логирование.
"""
import logging
import os
from datetime import datetime
from typing import Tuple, List, Optional
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.remote.webelement import WebElement
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
    TimeoutException, NoSuchElementException,
    StaleElementReferenceException,
    ElementNotInteractableException,
    ElementClickInterceptedException
)
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.keys import Keys

class BasePage:
    """Базовый класс для всех страниц веб-приложения"""
```

```

DEFAULT_BASE_URL = "https://the-internet.herokuapp.com"

def __init__(self, driver: WebDriver, base_url:
Optional[str] = None):
    self.driver = driver
    self.base_url = base_url or self.DEFAULT_BASE_URL
    self.wait = WebDriverWait(driver, 10)
    self.actions = ActionChains(driver)
    self.logger = logging.getLogger(self.__class__.__name__)

def open(self, url: str = ""):
    """Открытие страницы по относительному URL"""
    full_url = f"{self.base_url}/{url.lstrip('/')}"
    self.logger.info(f"Открытие страницы: {full_url}")
    self.driver.get(full_url)

def find_element(self, locator: Tuple[By, str], timeout: int
= 10) -> WebElement:
    """Поиск элемента с ожиданием"""
    try:
        wait = WebDriverWait(self.driver, timeout)
        element =
wait.until(EC.presence_of_element_located(locator))
        self.logger.debug(f"Элемент найден: {locator}")
        return element
    except TimeoutException:
        self.logger.error(f"Элемент не найден: {locator}")
        raise

def find_elements(self, locator: Tuple[By, str], timeout:
int = 10) -> List[WebElement]:
    """Поиск нескольких элементов"""
    try:
        wait = WebDriverWait(self.driver, timeout)
        elements =
wait.until(EC.presence_of_all_elements_located(locator))

```

```

        self.logger.debug(f"Найдено элементов:
{len(elements)}")

        return elements

    except TimeoutException:

        self.logger.warning(f"Элементы не найдены:
{locator}")

        return []

    def wait_for_element_visible(self, locator: Tuple[By, str],
    timeout: int = 10) -> WebElement:

        """Ожидание видимости элемента"""

        try:

            element = WebDriverWait(self.driver, timeout).until(
                EC.visibility_of_element_located(locator)
            )

            self.logger.debug(f"Элемент видим: {locator}")

            return element

        except TimeoutException:

            self.logger.error(f"Элемент не стал видимым:
{locator}")

            raise

    def wait_for_element_clickable(self, locator: Tuple[By,
    str], timeout: int = 10) -> WebElement:

        """Ожидание кликабельности элемента"""

        try:

            element = WebDriverWait(self.driver, timeout).until(
                EC.element_to_be_clickable(locator)
            )

            self.logger.debug(f"Элемент кликабелен: {locator}")

            return element

        except TimeoutException:

            self.logger.error(f"Элемент не стал кликабельным:
{locator}")

            raise

    def click(self, locator: Tuple[By, str], timeout: int = 10):

        """Клик по элементу"""

```

```

        try:
            element = self.wait_for_element_clickable(locator,
timeout)

            element.click()

            self.logger.info(f"Клик по элементу: {locator}")
        except (ElementNotInteractableException,
ElementClickInterceptedException,
            StaleElementReferenceException) as e:
            self.logger.error(f"Ошибка при клике: {e}")
            # Попытка клика через JavaScript
            element = self.find_element(locator, timeout)
            self.driver.execute_script("arguments[0].click();",
element)

    def input_text(self, locator: Tuple[By, str], text: str,
timeout: int = 10):
        """Ввод текста в поле"""
        element = self.wait_for_element_visible(locator,
timeout)
        element.clear()
        element.send_keys(text)
        self.logger.info(f"Ввод текста '{text}' в элемент:
{locator}")

    def get_text(self, locator: Tuple[By, str], timeout: int =
10) -> str:
        """Получение текста элемента"""
        element = self.wait_for_element_visible(locator,
timeout)
        text = element.text
        self.logger.debug(f"Текст элемента {locator}: {text}")
        return text

    def get_attribute(self, locator: Tuple[By, str], attribute:
str, timeout: int = 10) -> str:
        """Получение атрибута элемента"""
        element = self.find_element(locator, timeout)
        value = element.get_attribute(attribute)

```

```

        self.logger.debug(f"Атрибут {attribute} элемента
{locator}: {value}")
        return value

    def is_element_present(self, locator: Tuple[By, str],
timeout: int = 5) -> bool:
        """Проверка наличия элемента"""
        try:
            self.find_element(locator, timeout)
            return True
        except (TimeoutException, NoSuchElementException):
            return False

    def is_element_visible(self, locator: Tuple[By, str],
timeout: int = 5) -> bool:
        """Проверка видимости элемента"""
        try:
            self.wait_for_element_visible(locator, timeout)
            return True
        except TimeoutException:
            return False

    def scroll_to_element(self, locator: Tuple[By, str],
timeout: int = 10):
        """Прокрутка к элементу"""
        element = self.find_element(locator, timeout)

self.driver.execute_script("arguments[0].scrollIntoView(true);",
element)

        self.logger.debug(f"Прокрутка к элементу: {locator}")

    def accept_alert(self):
        """Принятие алерта"""
        alert = self.driver.switch_to.alert
        alert.accept()
        self.logger.debug("Алерт принят")

    def dismiss_alert(self):

```

```

        """Отклонение алерта"""
        alert = self.driver.switch_to.alert
        alert.dismiss()
        self.logger.debug("Алерт отклонен")

    def take_screenshot(self, name: str = "screenshot"):
        """Создание скриншота страницы"""
        os.makedirs("screenshots", exist_ok=True)
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = os.path.join("screenshots",
f"{name}_{timestamp}.png")
        self.driver.save_screenshot(filename)
        self.logger.info(f"Скриншот сохранен: {filename}")
        return filename

```

Page Objects

Для каждой страницы создан класс-наследник `BasePage` с локами-константами и методами взаимодействия. `HomePage` — переходы по ссылкам разделов; `InputsPage` — ввод/очистка/клавиши; `NotificationMessagesPage` — клик «Click here», чтение текста уведомления, закрытие крестиком, проверка текста на принадлежность ожидаемым вариантам. Наиболее нетривиальный объект — `GeolocationPage` (листинг ниже): в headless-браузерах реальное определение местоположения недоступно, поэтому перед кликом метод `mock_geolocation` подменяет `navigator.geolocation.getCurrentPosition` стабом с фиксированными координатами (детерминированно и одинаково в Chrome и Firefox, без системных разрешений геолокации).

```
# -*- coding: utf-8 -*-

"""Page Object страницы Geolocation (определение координат)."""
import json
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait
from .base_page import BasePage
from data.test_data import TestData


class GeolocationPage(BasePage):
    """Страница раздела Geolocation"""

    # Локаторы
    HEADING = (By.CSS_SELECTOR, "div.example h3")
    DEMO_TEXT = (By.ID, "demo")
    WHERE_AM_I_BUTTON = (By.XPATH, "//button[contains(., 'Where am I')]")
    LAT_VALUE = (By.ID, "lat-value")
    LONG_VALUE = (By.ID, "long-value")
    MAPS_LINK = (By.PARTIAL_LINK_TEXT, "See it on Google")

    def open_geolocation_page(self):
        """Открытие страницы Geolocation напрямую"""
```

```

self.open("geolocation")

def get_heading_text(self) -> str:
    """Текст заголовка страницы"""
    return self.get_text(self.HEADING)

def get_demo_text(self) -> str:
    """Текст абзаца #demo (приглашение либо координаты)"""
    return " ".join(self.get_text(self.DEMO_TEXT).split())

def mock_geolocation(self, latitude: float = None,
longitude: float = None):
    """Подмена navigator.geolocation.getCurrentPosition
стаб-координатами.

    В headless-браузерах реальное определение местоположения
недоступно,

    поэтому перед кликом подменяется браузерный API
Geolocation:

    любой запрос позиции мгновенно возвращает заданные
координаты

    (детерминированно в Chrome и Firefox, без системных
разрешений).
    """

    latitude = TestData.MOCK_LATITUDE if latitude is None
    else latitude

    longitude = TestData.MOCK_LONGITUDE if longitude is None
    else longitude

    payload = json.dumps({"coords": {"latitude": latitude,
"longitude": longitude}})

    self.driver.execute_script(
        "navigator.geolocation.getCurrentPosition = "
        "function(success){ success(" + payload + "); };"
    )

    self.logger.info(f"Geolocation подменен: lat={latitude},
long={longitude}")

def click_where_am_i(self):
    """Клик по кнопке 'Where am I?'"""

```

```

self.click(self.WHERE_AM_I_BUTTON)
self.logger.info("Выполнен клик 'Where am I?')

def wait_coordinates_shown(self, timeout: int = 10):
    """Ожидание отображения координат после клика"""
    WebDriverWait(self.driver, timeout).until(
        EC.presence_of_element_located(self.LAT_VALUE)
    )
    self.logger.info("Координаты отображены")

def get_latitude_text(self) -> str:
    """Текст широты"""
    return " ".join(self.get_text(self.LAT_VALUE).split())

def get_longitude_text(self) -> str:
    """Текст долготы"""
    return " ".join(self.get_text(self.LONG_VALUE).split())

def get_maps_link_href(self) -> str:
    """Ссылка 'See it on Google Maps' (атрибут href)"""
    return self.get_attribute(self.MAPS_LINK, "href")

def get_maps_link_text(self) -> str:
    """Текст ссылки на Google Maps"""
    return " ".join(self.get_text(self.MAPS_LINK).split())

```

Фикстуры PyTest (tests/conftest.py)

Параметры `--browser` и `--headless` регистрируются через `pytest_addoption` и доступны фикстурам. Фикстура `driver` создаёт свежий драйвер для каждого теста (полная изоляция) и закрывает его после. Auto-use-фикстура `screenshot_on_failure` по хуку `pytest_runtest_makereport` определяет падение теста и сохраняет скриншот страницы в момент падения (`screenshots/FAIL_<имя>.png`) — упрощает разбор падений. Остальные фикстуры поставляют готовые Page Objects.

```
# -*- coding: utf-8 -*-

"""Фикстуры pytest для UI-тестов the-internet.herokuapp.com."""

import logging
import pytest

from selenium.webdriver.remote.webdriver import WebDriver
from utils.driver_manager import DriverManager
from utils.logger import setup_logger
from utils.screenshot import ScreenshotManager
from pages.home_page import HomePage
from pages.inputs_page import InputsPage
from pages.notification_messages_page import NotificationMessagesPage
from pages.geolocation_page import GeolocationPage


# Настройка логирования
setup_logger()
logger = logging.getLogger(__name__)


def pytest_addoption(parser):
    """Параметры командной строки: браузер и режим headless"""
    parser.addoption(
        "--browser",
        action="store",
        default="chrome",
        choices=["chrome", "firefox", "edge"],
```

```

        help="Браузер: chrome, firefox или edge",
    )
    parser.addoption(
        "--headless",
        action="store_true",
        default=False,
        help="Запуск браузера в headless-режиме",
    )

@pytest.fixture(scope="session")
def browser(request):
    """Браузер из командной строки (--browser)"""
    return request.config.getoption("--browser")

@pytest.fixture(scope="session")
def headless(request):
    """Режим headless из командной строки (--headless)"""
    return request.config.getoption("--headless")

@pytest.fixture(scope="function")
def driver(browser, headless) -> WebDriver:
    """Фикстура для инициализации драйвера"""
    driver_manager = DriverManager(browser=browser,
    headless=headless)
    driver = driver_manager.create_driver()

    yield driver

    # Закрытие драйвера после теста
    driver_manager.quit_driver()

@pytest.fixture(scope="function")

```

```

def home_page(driver) -> HomePage:
    """Фикстура главной страницы"""
    return HomePage(driver)


@pytest.fixture(scope="function")
def inputs_page(driver) -> InputsPage:
    """Фикстура страницы Inputs"""
    return InputsPage(driver)


@pytest.fixture(scope="function")
def notification_messages_page(driver) ->
NotificationMessagesPage:
    """Фикстура страницы Notification Messages"""
    return NotificationMessagesPage(driver)


@pytest.fixture(scope="function")
def geolocation_page(driver) -> GeolocationPage:
    """Фикстура страницы Geolocation"""
    return GeolocationPage(driver)


@pytest.fixture(scope="function", autouse=True)
def screenshot_on_failure(request, driver):
    """Фикстура для создания скриншотов при падении тестов"""
    screenshot_manager = ScreenshotManager(driver)

    yield

    # Проверка, упал ли тест
    if hasattr(request.node, "rep_call") and
request.node.rep_call.failed:
        test_name = request.node.name
        screenshot_manager.take_screenshot(f"FAIL_{test_name}")

```

```
        logger.error(f"Тест {test_name} упал. Скриншот  
сохранен.")
```

```
@pytest.hookimpl(tryfirst=True, hookwrapper=True)  
def pytest_runtest_makereport(item, call):  
    """Хук для получения результатов теста"""  
    outcome = yield  
    rep = outcome.get_result()  
  
    # Сохраняем результат в атрибут теста  
    setattr(item, "rep_" + rep.when, rep)
```

Тесты (tests/)

Тесты сгруппированы по разделам в классы `TestInputs`, `TestNotificationMessages`, `TestGeolocation`; сценарии соответствуют таблице выше. Ниже — модуль `test_notification_messages.py`: навигация с главной, появление уведомления после клика, проверка случайного текста на принадлежность ожидаемым вариантам, закрытие крестиком (негативный сценарий) и устойчивость к серии кликов. Аналогично устроены `test_inputs.py` (ввод чисел, очистка, игнорирование букв, стрелки) и `test_geolocation.py` (приглашение, стаб-координаты, ссылка на карту).

```
# -*- coding: utf-8 -*-
"""UI-тесты раздела Notification Messages (flash-
уведомления)."""
import pytest
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait
from data.test_data import TestData

class TestNotificationMessages:
    """Тесты раздела Notification Messages"""

    @pytest.mark.smoke
    @pytest.mark.notifications
    def test_open_notification_messages_from_home(self,
home_page, notification_messages_page):
        """Переход в раздел Notification Messages с главной
страницы"""
        home_page.open_home_page()
        home_page.open_notification_messages()

        assert "notification_message" in
home_page.driver.current_url

        assert notification_messages_page.get_heading_text() ==
"Notifcation Message"

    @pytest.mark.smoke
    @pytest.mark.notifications
```

```

def test_notification_appears_after_click(self,
notification_messages_page):
    """После клика 'Click here' отображается flash-
уведомление"""
    notification_messages_page.open_notification_page()

    notification_messages_page.click_here()
    notification_messages_page.wait_flash_visible()

    assert notification_messages_page.is_flash_visible()

@pytest.mark.notifications
def test_notification_text_is_known(self,
notification_messages_page):
    """Текст уведомления – один из ожидаемых вариантов
сайта"""
    notification_messages_page.open_notification_page()

    notification_messages_page.click_here()
    notification_messages_page.wait_flash_visible()

    assert notification_messages_page.flash_text_is_known(),
(
    f"Неожиданный текст уведомления: "
    f"'{notification_messages_page.get_flash_text()}'"
)

@pytest.mark.negative
@pytest.mark.notifications
def test_notification_can_be_closed(self,
notification_messages_page):
    """Уведомление закрывается крестиком"""
    notification_messages_page.open_notification_page()
    notification_messages_page.wait_flash_visible()

    notification_messages_page.close_flash()

```

```

        WebDriverWait(notification_messages_page.driver,
10).until(

EC.invisibility_of_element_located(notification_messages_page.FL
ASH)

        )

        assert not notification_messages_page.is_flash_visible()


@pytest.mark.notifications
def test_notification_reappears_on_each_click(self,
notification_messages_page):
    """Каждый из нескольких кликов загружает новое
уведомление"""
    notification_messages_page.open_notification_page()

    for click_number in range(3):
        notification_messages_page.click_here()
        notification_messages_page.wait_flash_visible()

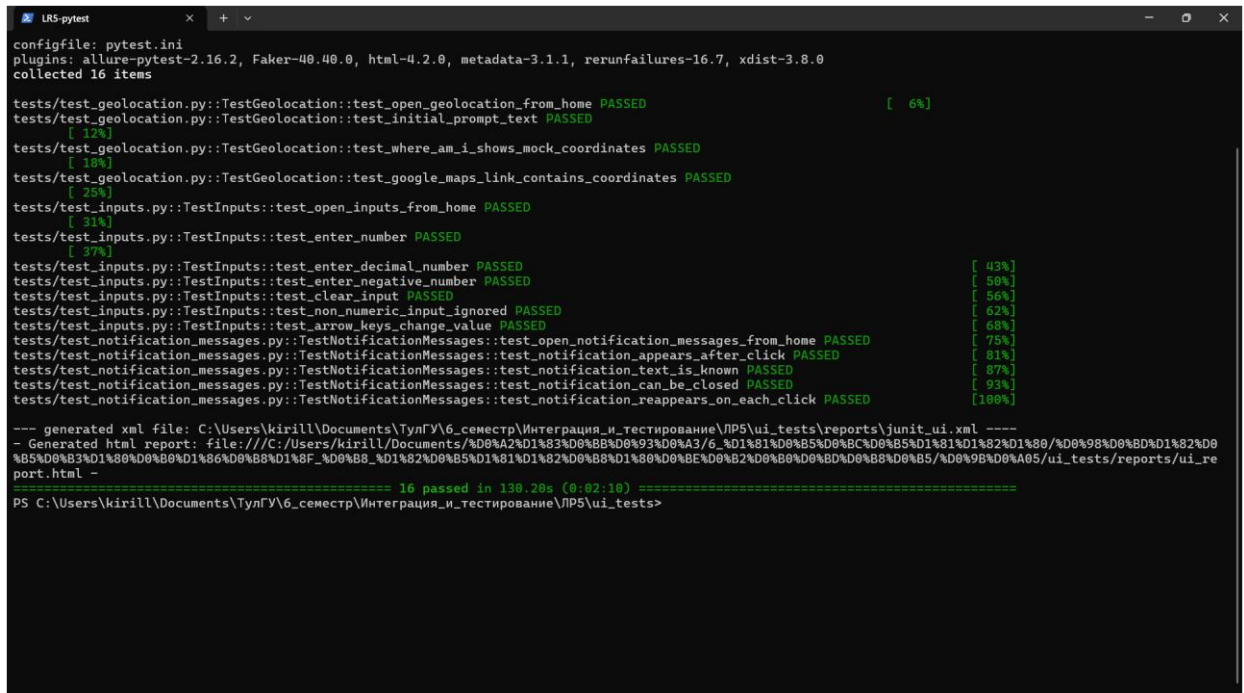
        assert
notification_messages_page.is_flash_visible(), (
            f"Уведомление не появилось после клика
№{click_number + 1}"
        )

        assert
notification_messages_page.flash_text_is_known()

```

Запуск тестов и результаты

Порядок запуска: из каталога LP5\ui_tests командой `py -m pytest tests/ --headless (Chrome)`; для Firefox — `py -m pytest tests/ --browser=firefox --headless`. Конфигурация `pytest.ini` автоматически генерирует HTML- и JUnit-отчёты и выполняет один автоповтор упавшего теста (`pytest-rerunfailures`). Для Allure — ключ `--alluredir=reports/allure_ui`. Полный текст вывода прогонов (Chrome и Firefox) сохранён в LP5\pytest_output.txt.



```
configfile: pytest.ini
plugins: allure-pytest-2.16.2, Faker-4.0.0, html-4.2.0, metadata-3.1.1, rerunfailures-16.7, xdist-3.8.0
collected 16 items

tests/test_geolocation.py::TestGeolocation::test_open_geolocation_from_home PASSED [ 6%]
tests/test_geolocation.py::TestGeolocation::test_initial_prompt_text PASSED [ 12%]
tests/test_geolocation.py::TestGeolocation::test_where_am_i_shows_mock_coordinates PASSED [ 18%]
tests/test_geolocation.py::TestGeolocation::test_google_maps_link_contains_coordinates PASSED [ 25%]
tests/test_inputs.py::TestInputs::test_open_inputs_from_home PASSED [ 31%]
tests/test_inputs.py::TestInputs::test_enter_number PASSED [ 37%]
tests/test_inputs.py::TestInputs::test_enter_decimal_number PASSED [ 43%]
tests/test_inputs.py::TestInputs::test_enter_negative_number PASSED [ 50%]
tests/test_inputs.py::TestInputs::test_clear_input PASSED [ 56%]
tests/test_inputs.py::TestInputs::test_non_numeric_input_ignored PASSED [ 62%]
tests/test_inputs.py::TestInputs::test_arrow_keys_change_value PASSED [ 68%]
tests/test_notification_messages.py::TestNotificationMessages::test_open_notification_messages_from_home PASSED [ 75%]
tests/test_notification_messages.py::TestNotificationMessages::test_notification_appears_after_click PASSED [ 81%]
tests/test_notification_messages.py::TestNotificationMessages::test_notification_text_is_known PASSED [ 87%]
tests/test_notification_messages.py::TestNotificationMessages::test_notification_can_be_closed PASSED [ 93%]
tests/test_notification_messages.py::TestNotificationMessages::test_notification_reappears_on_each_click PASSED [100%]

--- generated xml file: C:\Users\kirill\Documents\ТулГУ\6_семестр\Интеракция_и_тестирование\LP5\ui_tests\reports\junit-ui.xml ---
- Generated html report: file:///C:/Users/kirill/Documents/%D0%A2%D1%83%D0%B8%D0%93%D0%A3/6_%D1%81%D0%B5%D0%BC%D0%B5%D1%81%D1%82%D1%80/%D0%98%D0%BD%D1%82%D0%B5%D0%B0%D1%86%D0%B8%D1%8F_%D0%B8_%D1%82%D0%B5%D1%81%D1%82%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D0%B5/%D0%98%D0%A05/ui_tests/reports/ui_report.html -
===== 16 passed in 130.20s (0:02:10) =====
PS C:\Users\kirill\Documents\ТулГУ\6_семестр\Интеракция_и_тестирование\LP5\ui_tests>
```

Рисунок 5 — Прогон в видимом окне терминала: collected 16 items, 16 passed

Результат: 16 тестов — 16 passed, 0 failed в обоих браузерах. Chrome (headless): 16 passed за ~2 мин (в одном прогоне геолокационный тест единожды ушёл в автоповтор из-за тайминга первого срабатывания стаба и прошёл на повторе — «16 passed, 1 rerun»). Firefox (headless, geckodriver через webdriver-manager): 16 passed за ~3,7 мин. Задание «не менее 3 сценариев» выполнено с запасом: 16 сценариев, из них негативных — 2 (ТС-INP-06, ТС-NOT-04), дымовых — 6 (маркер smoke).

Отчётность

`pytest-html` (addopts: `--html=reports/ui_report.html --self-contained-html --junitxml=reports/junit_ui.xml`) формирует автономный HTML-отчёт со сводкой, параметрами окружения и таблицей результатов с раскрываемыми деталями (включая перехваченные логи действий страниц). JUnit XML предназначен для CI. Дополнительно настроен

Allure: прогон с `--alluredir=reports/allure_ui` создаёт результаты, allure generate строит по ним отчёт (Overview со сводкой по severity/длительности, Suites по классам, шаги и скриншоты в деталях тестов).

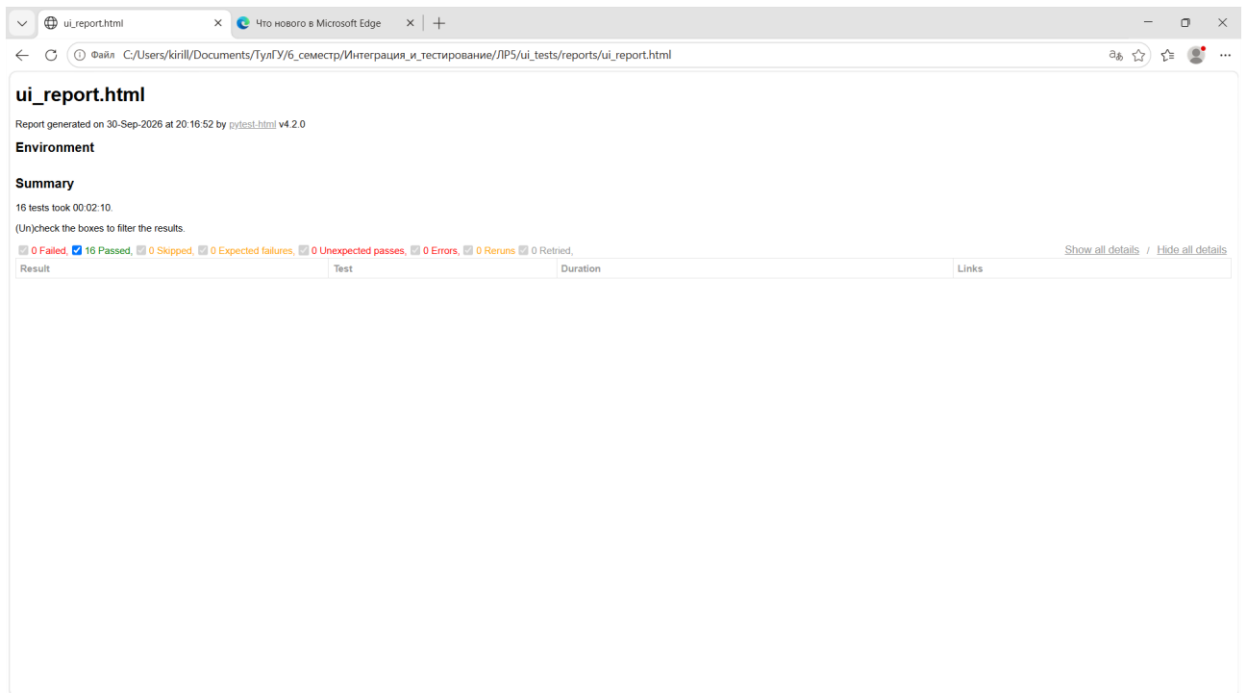


Рисунок 6 — HTML-отчёт pytest-html в браузере (Summary: 16 passed)

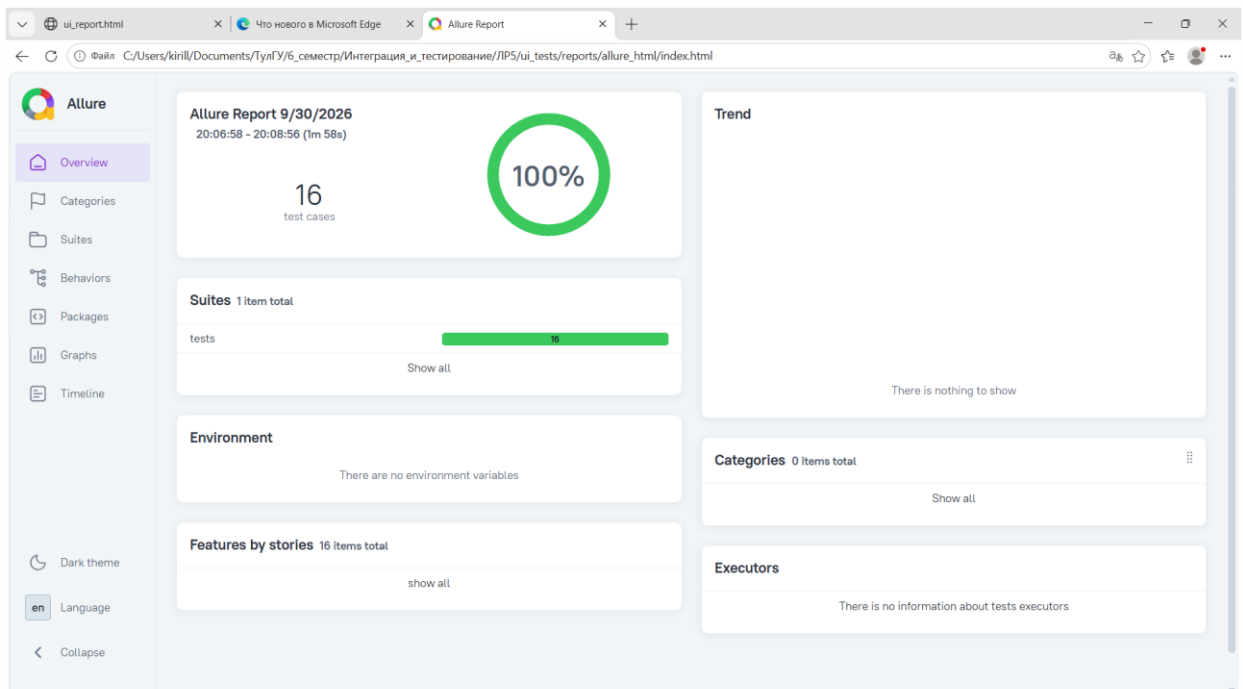


Рисунок 7 — Отчёт Allure (Overview) в браузере

Анализ найденных проблем и предложения по улучшению

1) Перекрывание элементов: кнопка-крестик flash-уведомления на Notification Messages частично перекрывается уголковым баннером

«Fork me on GitHub» — Selenium получал `ElementClickInterceptedException`. Решение: в `BasePage.click` добавлен fallback-клик через JavaScript при перехвате/недоступности элемента; тест стал устойчивым без изменения локатора. 2) Нестабильные локаторы: на странице `Inputs` заголовок `h3` находится вне контейнера `div.example` (в отличие от двух других разделов) — селектор `div.example h3` находил заголовок только на части страниц. Вывод: локаторы не должны зависеть от случайных особенностей вёрстки; использован прямой селектор `h3`. 3) Случайность ответа: текст flash-уведомления выбирается сайтом случайно из двух вариантов (включая опечатку «unsuccesful» на самом сайте) — жёсткое сравнение текста делало бы тест флакующим; проверка выполняется на принадлежность множеству ожидаемых вариантов. 4) Headless и геолокация: реальное определение координат в headless-браузере недоступно (и недетерминированно) — введён стаб `navigator.geolocation` с фиксированными координатами, что сделало сценарии `Geolocation` детерминированными и одинаковыми в обоих браузерах. 5) Тайминги: единичные гонки старта (геолокационный тест при первом прогоне) гасятся автоповтором (`pytest-rerunfailures`, 1 повтор). Предложения по улучшению: параллельный запуск тестов (`pytest-xdist`, ключ `-n auto` — включён в зависимости, строка `addopts` закомментирована), вынос ожидаемых текстов в `data/test_data.py` (уже сделано) и расширение набора разделами других вариантов, CI-прогон с публикацией Allure (`junit_ui.xml` уже совместим с GitLab CI).

Выводы

Автоматизировано UI-тестирование разделов `Geolocation`, `Inputs` и `Notification Messages` сайта `The Internet` (вариант 9): 16 сценариев (в 4,3 раза больше минимума), выполненных в обоих браузерах — Chrome и Firefox в headless-режиме, 16 passed / 0 failed. Реализован полный паттерн POM: `BasePage` с явными ожиданиями, логированием и JS-fallback кликов; по Page Object на раздел; фикстуры с параметрами браузера и autouse-скриншотами падений; данные вынесены в `data/test_data.py`. Настройки отчётности: `pytest-html` + JUnit XML автоматически на каждом прогоне, Allure — по ключу `--alluredir` с генерацией отчёта `allure generate`. Найденные при автоматизации проблемы (перекрытие клика баннером,

нестабильные локаторы, случайный текст уведомлений, недоступность реальной геолокации в headless) решены инженерными приёмами — JS-клик, независимые от вёрстки селекторы, проверка на множество ожидаемых текстов, детерминированный стаб navigator.geolocation; единичные тайминговые гонки гасятся автоповтором. Проект переиспользуем: добавление нового раздела сайта сводится к созданию Page Object и модуля тестов.